```
Removing intermediate container 47ec46df6236
 ---> ea5d358cb673
Successfully built ea5d358cb673
Successfully tagged myapp_web:latest
$ docker-compose up -d --force-recreate web
Recreating myapp_web_1 ... done
```

Next we need to install RSpec, setting up its file structure:

```
$ docker-compose exec web bin/rails generate rspec:install
      create  .rspec
      create  spec
      create  spec/spec_helper.rb
      create  spec/rails_helper.rb
```

With RSpec set up, we're able to run the specs like so:

```
$ docker-compose exec web bin/rails spec
```

However, as you'd expect, this reports that we have no specs currently:

```
No examples found.

Finished in 0.00509 seconds (files took 0.30574 seconds to load)
0 examples, 0 failures
```

Let's take RSpec for a proper test-drive by creating…

Our First Test

Having installed RSpec in our project, it's not very satisfying to see zero tests running. Let's rectify that right now by creating our first test, so that we can see some actual test code running.

Let's generate a spec for our `User` model:

```
$ docker-compose exec web bin/rails generate rspec:model user
      create  spec/models/user_spec.rb
```

Linux Users: File Ownership



Remember to `chown` the files we've generated so you can edit them (see File Ownership and Permissions, on page 199):

```
$ sudo chown <your_user>:<your_group> -R .
```

Open the newly created `spec/models/user_spec.rb` file in your editor. This isn't a book about testing—we just need a basic test to show RSpec is working like we expect. The following should do the trick: